

AIMS 5740

Generative Artificial Intelligence

(2026 Term 2)



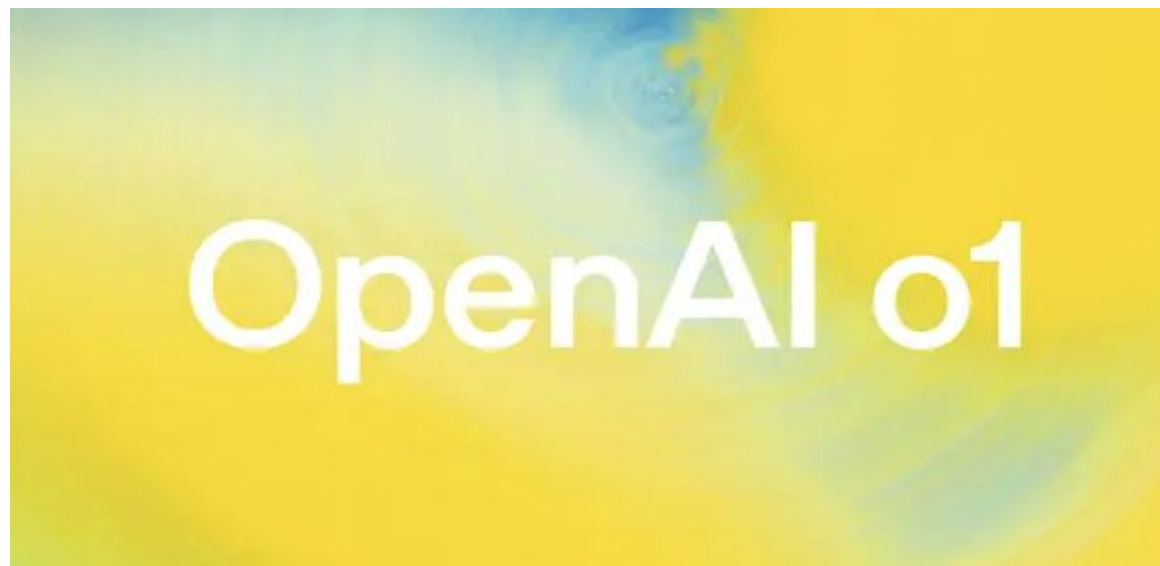
Computer Science & Engineering
The Chinese University of Hong Kong

Announcement

- Course Schedule:
 - 1) April 8: Test-time scaling & GRPO (by Dr. Li Yafu)
 - 2) April 15: project presentation
 - 3) April 26: project report due

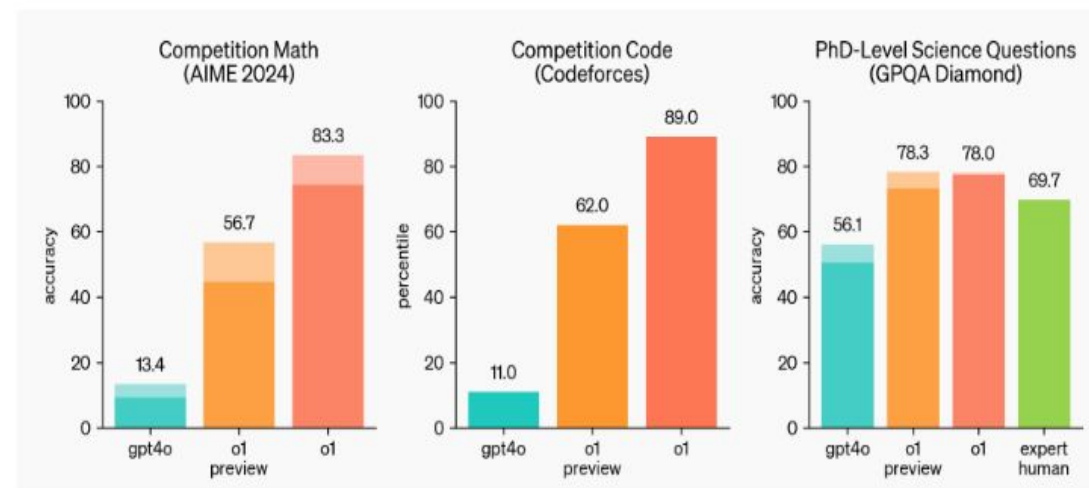
LLMs: Reasoning Models

New large language models trained with reinforcement learning to perform complex reasoning.

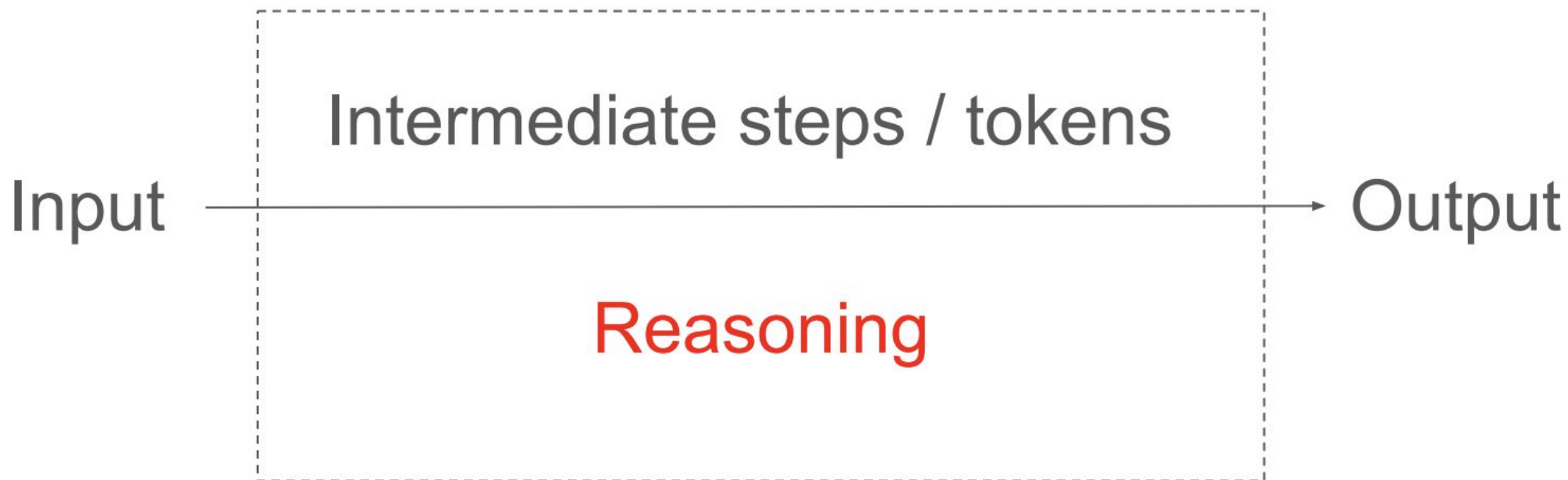


LLMs: Reasoning Models (2)

Through reinforcement learning, o1 learns to hone its chain of thought and refine the strategies it uses.



What is LLM Reasoning?



The first ever paper which uses natural language to describe intermediate reasoning steps: Ling et al. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. ACL 2017. Before this paper, intermediate steps were described by symbolic languages (see the neural-symbolic reasoning literature, including my early work).

What is LLM Reasoning?

What is the output when concatenating the last letter of each word in “artificial intelligence”?

No reasoning

The answer is “le”.

Reasoning



The last letter of “artificial” is “l”. The last letter of “intelligence” is “e”. Concatenating “l” and “e” leads to “le”. So the answer is “le”.

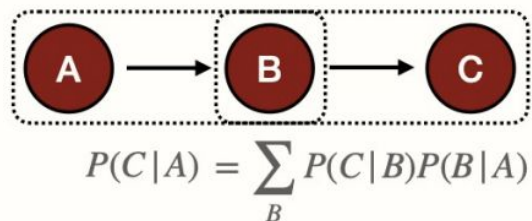
This synthetic last-letter-concatenation task was motivated from my early work: Neuro-symbolic program synthesis. arXiv preprint 1611.01855 (2016). Joint work with Parisotto et al. At that time, we solved similar problems by automatic program synthesis.

Why Thinking Step-by-Step can help ?

- ❖ Why adding **Intermediate Steps** can help LLMs, which, after all, are merely **Statistical Inference models**, to be better in answering difficult questions ?

Hypothesis from B. Prystawski, M.Y. Li, N.D.Goodman:

Step-by-step reasoning lets (humans/LMs) chain together local inferences between variables they have seen together a lot in order to support longer-distance inferences



“Our results suggest that chain-of-thought reasoning is useful for language models because:

1) direct prediction is inaccurate for some inferences because the relevant variables are rarely seen together in training and

2) chain-of-thought reasoning improves estimation by incrementally chaining local statistical dependencies that are observed frequently in training.

We also find that the combination of locally structured training data and reasoning with self-generated intermediate variables yields much greater data efficiency than training on data containing all variables.”

References:

- Ben Prystawski, Michael Y. Li, Noah D. Goodman, “Why think step by step ? Reasoning emerges from the Locality of Experience,” NeurIPS 2023, <https://neurips.cc/virtual/2023/oral/73821>, <https://neurips.cc/media/neurips-2023/Slides/73821.pdf>,
- Noah Goodman, “Learning to Reason, Insights from Language Modeling,” MIT Center for Brains, Minds+ Machines (MITCBMM) seminar, <https://www.youtube.com/watch?v=YR9FztOF0R8>,
- S.C.Y. Chan et al., “Data Distributional Properties Drive Emergent In-Context Learning in Transformers,” NeurIPS 2022.

Why Thinking Step-by-Step can help ?

Theoretical Results by Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma, “Chain of Thought Empowers Transformers to Solve Inherently Serial Problems,” ICLR 2024

TL;DR: *We show both theoretically and empirically transformers with polynomial steps of CoT can simulate polysize circuits and thus are strictly more expressive than transformers without CoT.*

- ❖ **Transformer generating Intermediate Steps** can solve any Inherently serial problem as long as its depth exceeds a constant threshold
- ❖ **Transformer generating direct answers** either requires a huge depth to solve or cannot solve at all

Why Thinking Step-by-Step can help ?

- For any problems solvable by boolean circuits of size T , **constant-size transformers** can solve it by generating $O(T)$ intermediate tokens
- If directly generating final answers, either requires a huge depth or cannot solve at all

Joint work Li et al. [Chain of Thought Empowers Transformers to Solve Inherently Serial Problems](#). ICLR 2024. “Chain of thought (CoT)” in this work just means “CoT reasoning”. Don’t mess it up with our “CoT prompting”.

LLMs are ready to reason

Pretrained LLMs cannot reason without further prompting engineering or finetuning

WRONG

LLMs are ready to reason

I have 3 apples. My dad has 2 more apples than me. How many apples do we have in total?

5 apples. (Greedy Decoding)

I have 3 apples, my dad has 2 more apples than me, so he has 5 apples. $3+5=8$.

We have 8 apples in total.

You have 3 apples, your dad has 2 more apples than you, so he has 5 apples. $3+5=8$.

The answer is 5.

How to “teach” / lead / force LLMs to Reason ?

- ❖ Type 1 Approaches: Using some Prompting techniques for the LLM, e.g. Chain-of-Thought (CoT) prompting
- ❖ Type 2 Approaches: Make LLM perform Search and Selection from Multiple Candidates
 - => Increase the **WIDTH** to explore solution space
- ❖ Type 3 Approaches: Make LLM perform Iterative self-improvement
 - => Increase the **DEPTH** to reach the final solution

Common Theme:

Make / Lead / Force LLM to use more Token Budget (i.e. “Think” more) before reaching the final solution !

i.e. Trigger/ Make the LLM to perform “System 2” like behavior of Human!

Chain-of-Thought (CoT) Prompting

- Asking the model to reason about its answer can improve its performance for few-shot in-context learning
- **Chain-of-thought prompting** provides such reasoning in the in-context examples

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

- But the model does better even if you just prompt it to reason step-by-step

Chain-of-Thought (CoT) Prompting

Q: "Elon Musk"

A: the last letter of "Elon" is "n". the last letter of "Musk" is "k". Concatenating "n", "k" leads to "nk". so the output is "nk".

reasoning process

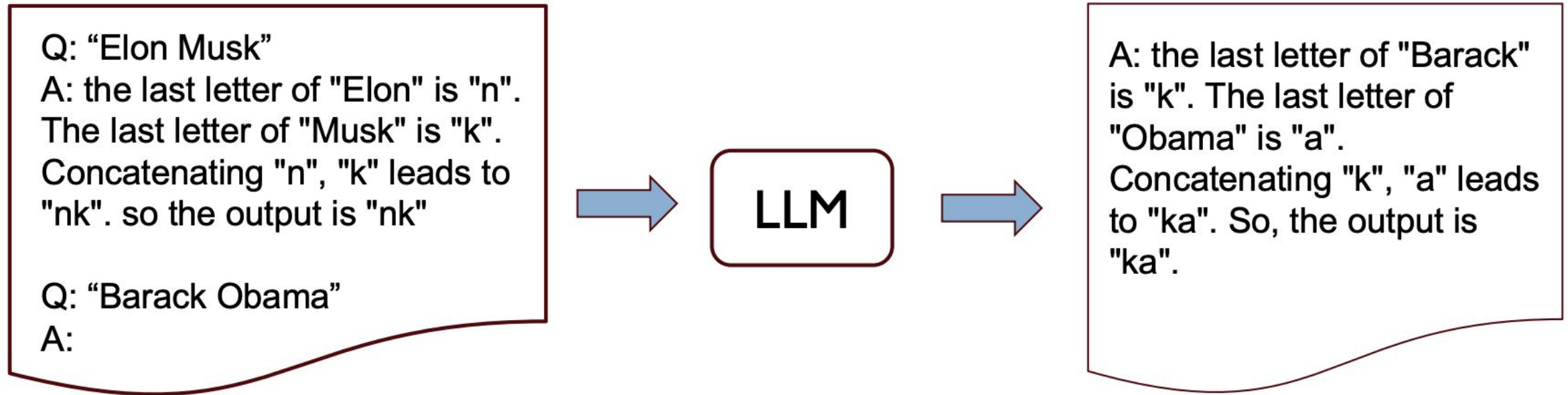
Q: "Bill Gates"

A: the last letter of "Bill" is "l". the last letter of "Gates" is "s". Concatenating "l", "s" leads to "ls". so the output is "ls".

Q: "Barack Obama"

A: the last letter of "Barack" is "k". the last letter of "Obama" is "a". Concatenating "k", "a" leads to "ka". so the output is "ka".

Teaching LLM to Reason via Chain of Thought (CoT) Prompting



J. Wei et al. [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#). NeurIPS 2022.

Dataset: GSM8K

Following the work by Ling et al 2017, Cobbe et al 2021 in OpenAI built a much larger math word problem dataset (GSM8K) with natural language rationales, and used it to finetune GPT3

Problem: Ali is a dean of a private school where he teaches one class. John is also a dean of a public school. John has two classes in his school. Each class has $\frac{1}{8}$ the capacity of Ali's class which has the capacity of 120 students. What is the combined capacity of both schools?

Solution: Ali's class has a capacity of 120 students. Each of John's classes has a capacity of $120/8 = 15$ students. The total capacity of John's two classes is $15 \text{ students} * 2 \text{ classes} = 30 \text{ students}$. The combined capacity of the two schools is $120 \text{ students} + 30 \text{ students} = 150 \text{ students}$.

Final answer: 150



Cobbe et al. Training Verifiers to Solve Math Word Problems. [arXiv:2110.14168](https://arxiv.org/abs/2110.14168) [cs.LG]. 2021

Zero-shot CoT

- Asking the model to reason about its answer can improve its performance for few-shot in-context learning
- **Chain-of-thought prompting** provides such reasoning in the in-context examples

	MultiArith	GSM8K
Zero-Shot	17.7	10.4
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	78.7	40.7
Few-Shot-CoT (2 samples)	84.8	41.3

- But the model does better even if you just prompt it to reason step-by-step

Zero-shot CoT still performs worse than few-shot CoT

	MultiArith	GSM8K
Zero-Shot	17.7	10.4
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	78.7	40.7
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	93.0	48.7
Zero-Plus-Few-Shot-CoT (8 samples) (*2)	92.8	51.5

How to improve CoT performance w/o manually labeling exemplars?

■ Pros and Cons of Prompting

Pros: Simple and works

Cons:

CoT prompting needs task-specific examples

“Let’s think step by step” is generic, but performs much worse than few-shot

Pros and Cons of Prompting

Prompting approaches are actually weird

When asking someone a question —

will you first show similar problems/solutions before asking?

or, at the end of you question, will you have to say “let’s think step by step”?

Of course not!

Issues with Chain-of-Thought approach

- ❖ Generating intermediate steps (i.e. CoT approach) are helpful BUT are they producing the correct direct answers (i.e. being an unbiased estimator) ?
 - ▶ After all, LLMs is still (merely) a probabilistic, inference model predicting the next token !

What LLM does in decoding:

$$\arg \max \mathbb{P}(\text{reasoning path, final answer} | \text{problem})$$

What we want:

$$\arg \max \mathbb{P}(\text{final answer} | \text{problem})$$



Issues with Chain-of-Thought approach

What we want:

$$\arg \max \mathbb{P}(\text{final answer}|\text{problem})$$

What LLM does in decoding:

$$\arg \max \mathbb{P}(\text{reasoning path, final answer}|\text{problem})$$

Luckily:

$$\begin{aligned} & \arg \max \mathbb{P}(\text{final answer}|\text{problem}) \\ &= \sum_{\text{reasoning path}} \mathbb{P}(\text{reasoning path, final answer}|\text{problem}) \end{aligned}$$

Thus, we can just compute the sum !

How to compute the sum then ? **Via Sampling !**

But how to reduce Sampling Variance so that we can get the right answer faster ?

Generate Multiple Solutions then Pick One

- ❖ Should NOT limit the LLM to generate only one solution per problem
- ❖ Tell LLM to explore, examine and compare multiple solutions which allows it to possibly recover from mistakes in a single generation (i.e. only a single CoT path).
Two different variants of this approach:
 - Generate **multiple candidate End-to-End solutions** per problem
 - Generate **multiple candidate “Next Steps”** given the current **partial** CoT path (i.e. “partial”, incomplete thought).

Challenge:

- ❖ How to select the **“Best”** choice among multiple candidates ?
 - In most cases, NO oracle scorer is available for the LLM at inference time

Self-Consistency Checks

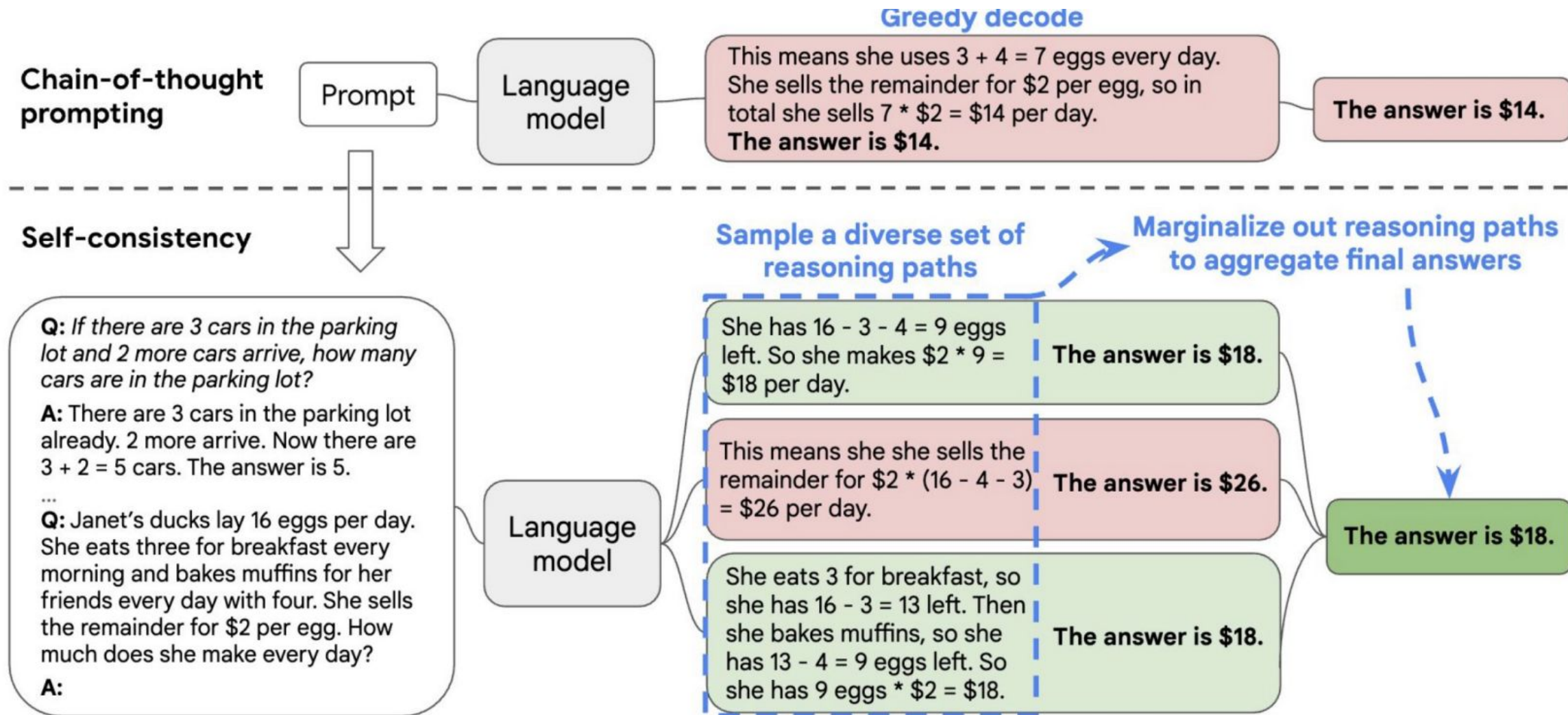


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using example chains of thought; (2) sample from the language model's decoder to generate a diverse set of reasoning paths; and (3) choose the most consistent answer using the majority/plurality vote.

Select the Response w/ the most Consistent Solutions

[Question] Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 per egg. How much does she make every day?

Sampled responses:

Response 1: She has $16 - 3 - 4 = 9$ eggs left. So she makes $\$2 * 9 = \boxed{\$18}$ per day.

Response 2: This means she she sells the remainder for $\$2 * (16 - 4 - 3) = \boxed{\$26}$ per day.

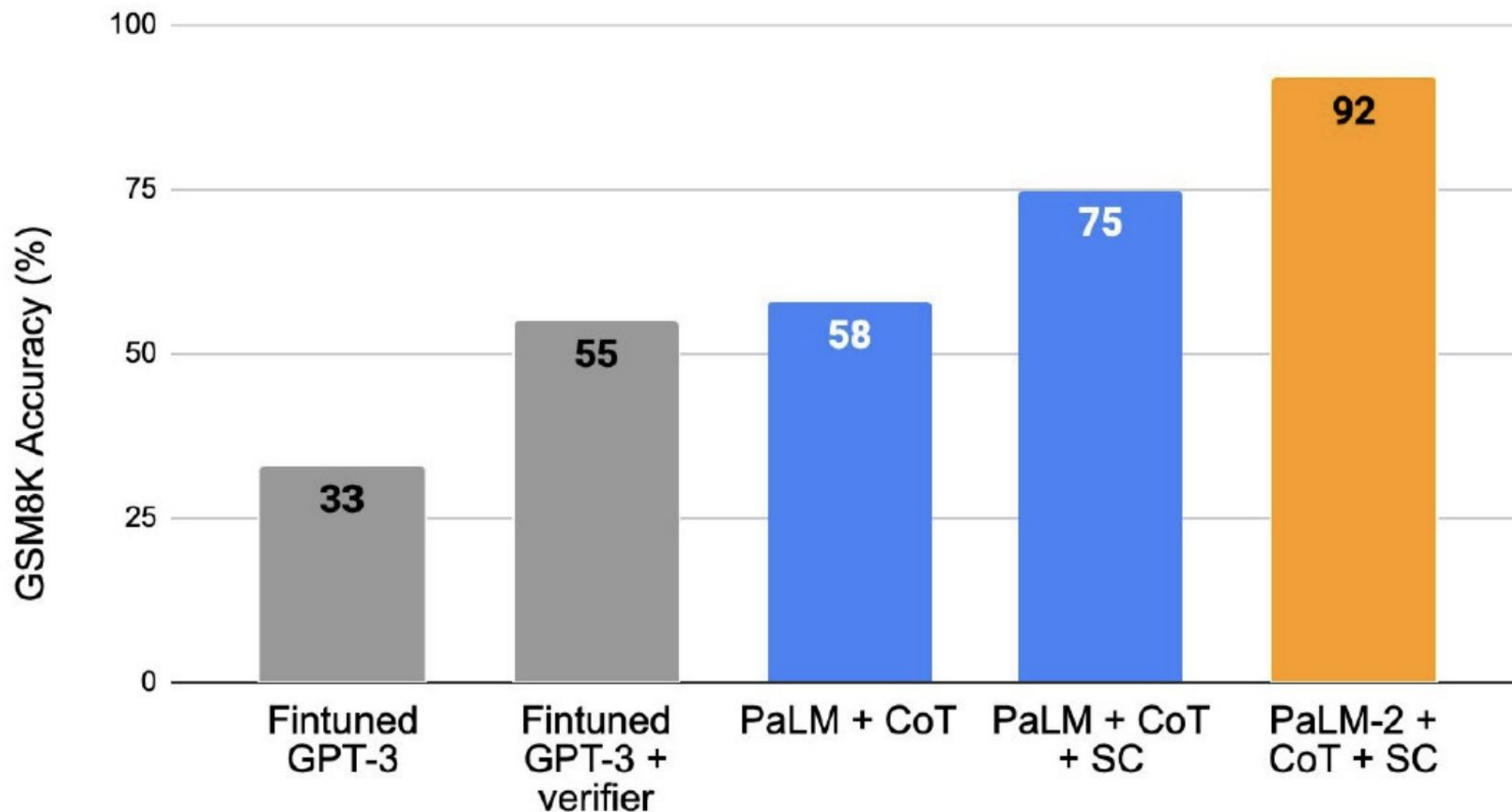
Response 3: She eats 3 for breakfast, so she has $16 - 3 = 13$ left. Then she bakes muffins, so she has $13 - 4 = 9$ eggs left. So she has $9 \text{ eggs} * \$2 = \boxed{\$18}$.

Most frequent answer is: 18
(Not most frequent reasoning path!)



Self-Consistency Checks: Results

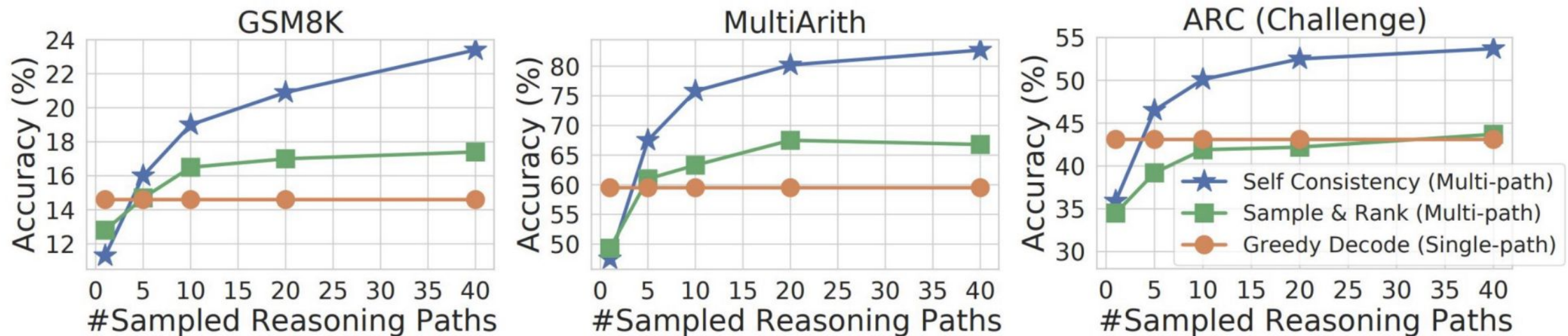
GSM8K SOTA with only 8 examples



Self-Consistency Checks: Results

Method		AddSub	MultiArith	ASDiv	AQuA	SVAMP	GSM8K
Previous SoTA		94.9^a	60.5 ^a	75.3 ^b	37.9 ^c	57.4 ^d	35 ^e / 55 ^g
UL2-20B	CoT-prompting	18.2	10.7	16.9	23.6	12.6	4.1
	Self-consistency	24.8 (+6.6)	15.0 (+4.3)	21.5 (+4.6)	26.9 (+3.3)	19.4 (+6.8)	7.3 (+3.2)
LaMDA-137B	CoT-prompting	52.9	51.8	49.0	17.7	38.9	17.1
	Self-consistency	63.5 (+10.6)	75.7 (+23.9)	58.2 (+9.2)	26.8 (+9.1)	53.3 (+14.4)	27.7 (+10.6)
PaLM-540B	CoT-prompting	91.9	94.7	74.0	35.8	79.0	56.5
	Self-consistency	93.7 (+1.8)	99.3 (+4.6)	81.9 (+7.9)	48.3 (+12.5)	86.6 (+7.6)	74.4 (+17.9)
GPT-3 Code-davinci-001	CoT-prompting	57.2	59.5	52.7	18.9	39.8	14.6
	Self-consistency	67.8 (+10.6)	82.7 (+23.2)	61.9 (+9.2)	25.6 (+6.7)	54.5 (+14.7)	23.4 (+8.8)
GPT-3 Code-davinci-002	CoT-prompting	89.4	96.2	80.1	39.8	75.8	60.1
	Self-consistency	91.6 (+2.2)	100.0 (+3.8)	87.8 (+7.6)	52.0 (+12.2)	86.8 (+11.0)	78.0 (+17.9)

Self-Consistency Checks: Results



- Sample-and-Rank baseline: select the response with the highest log probability
- Self-Consistency performance scales much better than probability-based ranking
 - Unless the model is trained to be a good verifier (will cover later)

Self-Consistency Checks: Results

	GSM8K	MultiArith	SVAMP	ARC-e	ARC-c
CoT (Wei et al., 2022)	17.1	51.8	38.9	75.3	55.1
Ensemble (3 sets of prompts)	18.6 $\pm$ 0.5	57.1 $\pm$ 0.7	42.1 $\pm$ 0.6	76.6 $\pm$ 0.1	57.0 $\pm$ 0.2
Ensemble (40 prompt permutations)	19.2 $\pm$ 0.1	60.9 $\pm$ 0.2	42.7 $\pm$ 0.1	76.9 $\pm$ 0.1	57.0 $\pm$ 0.1
Self-Consistency (40 sampled paths)	27.7 $\pm$ 0.2	75.7 $\pm$ 0.3	53.3 $\pm$ 0.2	79.3 $\pm$ 0.3	59.8 $\pm$ 0.2

Beam size / Self-consistency paths		1	5	10	20	40
AQuA	Beam search decoding (top beam)	23.6	19.3	16.1	15.0	10.2
	Self-consistency using beam search	23.6	19.8 $\pm$ 0.3	21.2 $\pm$ 0.7	24.6 $\pm$ 0.4	24.2 $\pm$ 0.5
	Self-consistency using sampling	19.7 $\pm$ 2.5	24.9 $\pm$ 2.6	25.3 $\pm$ 1.8	26.7 $\pm$ 1.0	26.9 $\pm$ 0.5
MultiArith	Beam search decoding (top beam)	10.7	12.0	11.3	11.0	10.5
	Self-consistency using beam search	10.7	11.8 $\pm$ 0.0	11.4 $\pm$ 0.1	12.3 $\pm$ 0.1	10.8 $\pm$ 0.1
	Self-consistency using sampling	9.5 $\pm$ 1.2	11.3 $\pm$ 1.2	12.3 $\pm$ 0.8	13.7 $\pm$ 0.9	14.7 $\pm$ 0.3

- ❖ Beam search: Keep Top-K paths with the highest probabilities in the Decoding process
- ❖ Ensemble baselines: apply greedy decoding for all prompt variants of a problem
- ❖ Self-Consistency (SC) uses sampling scales with more samples:
 - The sampling method needs to ensure response diversity, e.g., using a high temperature, nucleus sampling etc.

Why does Self-Consistency work ?

$$\begin{aligned} & \arg \max \mathbb{P}(\text{answer}|\text{problem}) && \text{(find the answer with the maximum probability)} \\ = & \sum_{\text{rationale}} \mathbb{P}(\text{answer, rationale}|\text{problem}) && \text{(sum over all latent reasoning paths)} \\ \approx & \frac{\text{frequency of the answer}}{\text{total number of sampled responses}} && \text{(approximate the sum by sampling)} \\ \propto & \text{frequency of the answer} && \text{(ignore the common factor of the same size)} \end{aligned}$$

Thus, $\arg \max \mathbb{P}(\text{answer}|\text{problem}) \approx$ selecting the most frequent answer.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, Denny Zhou.
[Self-Consistency Improves Chain of Thought Reasoning in Language Models](#). ICLR 2023.

Why does Self-Consistency work ?

Self-consistency is the empirical implementation of marginalization. Don't be superficial to interpret it as majority voting!



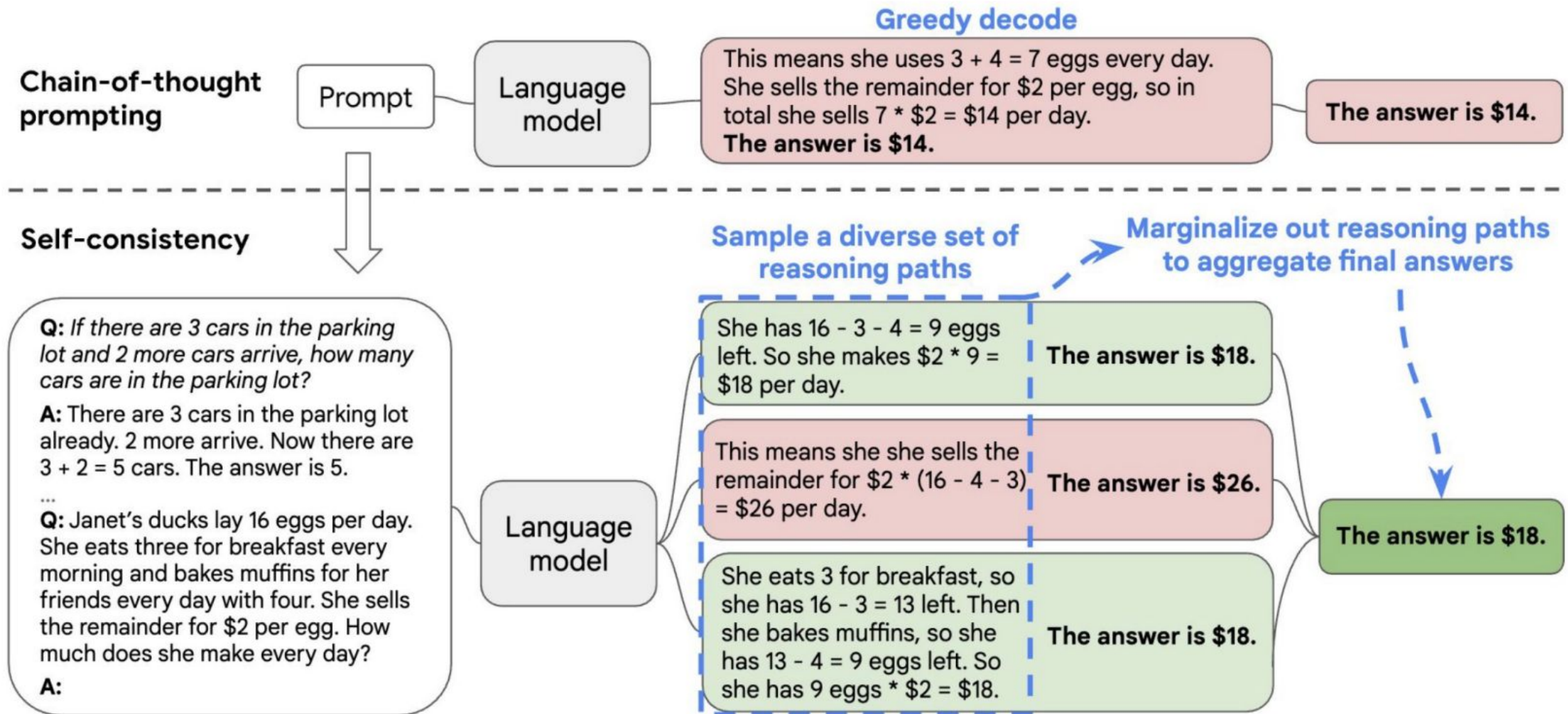
Self-consistency $\arg \max \mathbb{P}(\text{answer}|\text{problem})$

Chain-of-thought $\arg \max \mathbb{P}(\text{answer}, \text{rationale}|\text{problem})$

When there is no reasoning path, we don't need self-consistency, since we can then directly choose the most likely answer based on the given probabilities!

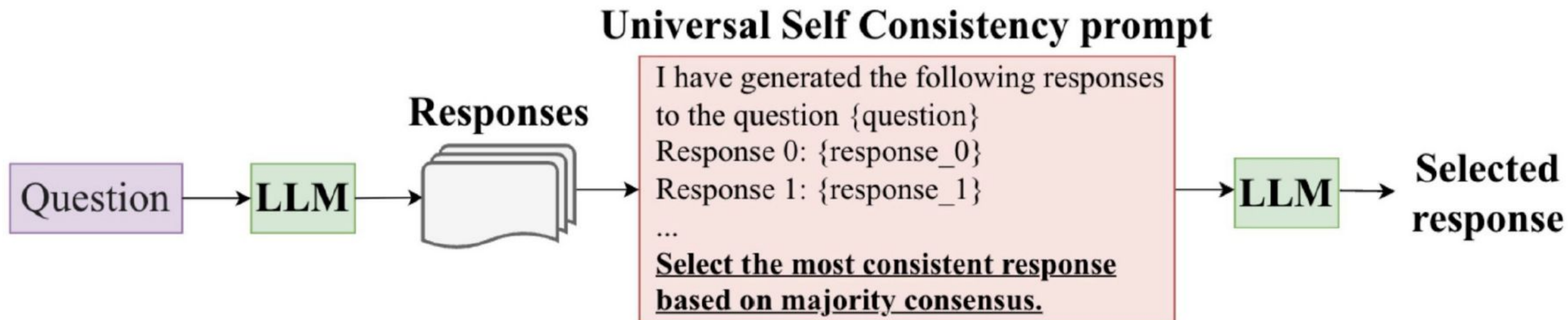


Limitation of SC: require an answer extraction



Can we enable consistency-based decoding for **free-form generation**?

Universal Self-Consistency (USC)



Universal Self-Consistency Check

[Question] Where do people drink less coffee than they do in Mexico?

Response 1: ... Some examples include Japan, China and the United Kingdom.

It is important to note that coffee consumption can vary among individuals within these countries, and preferences can change depending on different factors such as...

Response 2: People in countries like Japan, China, and India typically drink less coffee than they do in Mexico...

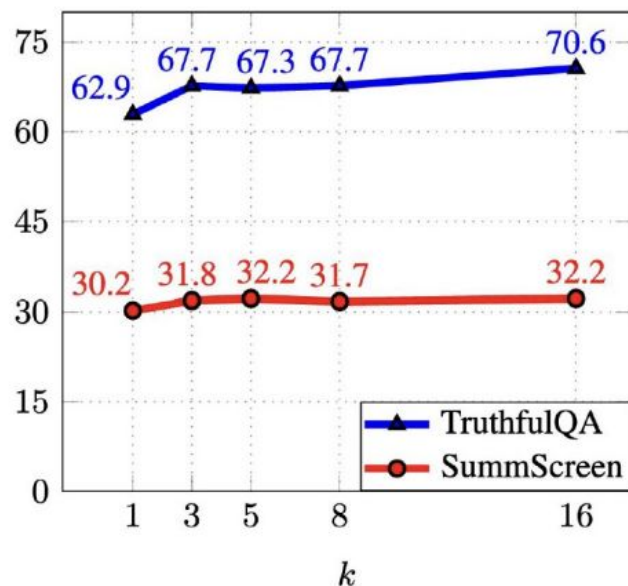
Response 3: There are several countries where people generally drink less coffee compared to Mexico. Some of these countries include:

1. Japan:...
2. China...
3. Saudi Arabia...
4. India...

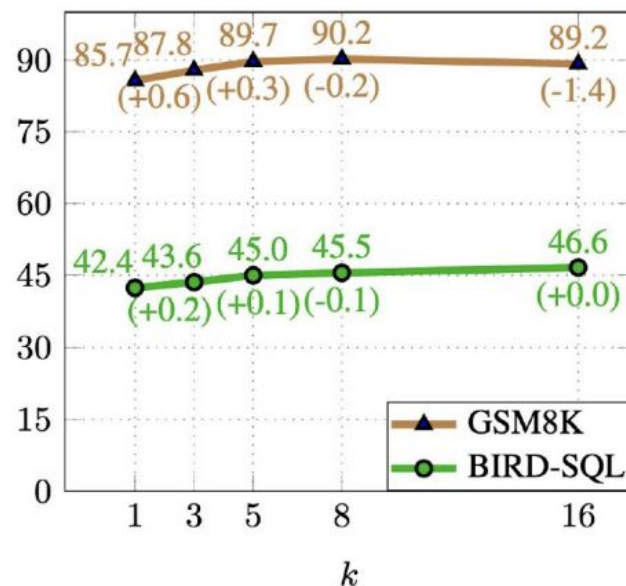
...

The most consistent response: 2

Universal Self-Consistency Check: Results



(a) Results on open-ended generation.



(b) Results on GSM8K and BIRD-SQL. The top numbers are USC accuracies, and the bottom numbers are the differences to SC accuracies.

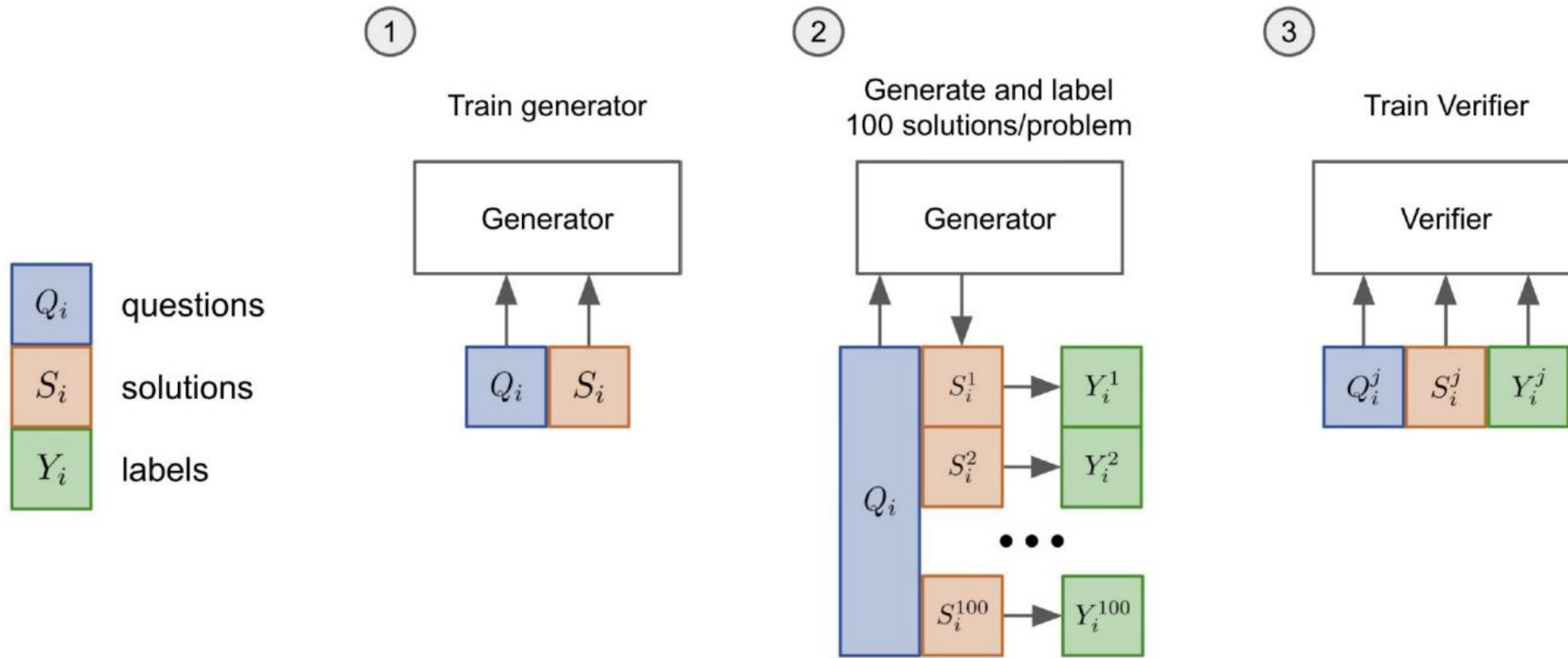
- Universal Self-Consistency (USC) improves the performance on open-ended generation (summarization, QA), where the original self-consistency is inapplicable.
- USC matches self-consistency performance on math reasoning and coding.
 - USC does not require answer extraction and code execution.
- USC performance is bounded by the long-context capability.

Best-of-N with Learned Verifier

STEPS:

1. Fine-Tune Verifier LM
2. Sample N answers
3. Select the Best answer according to the Verifier

Best-of-N with Learned Verifier

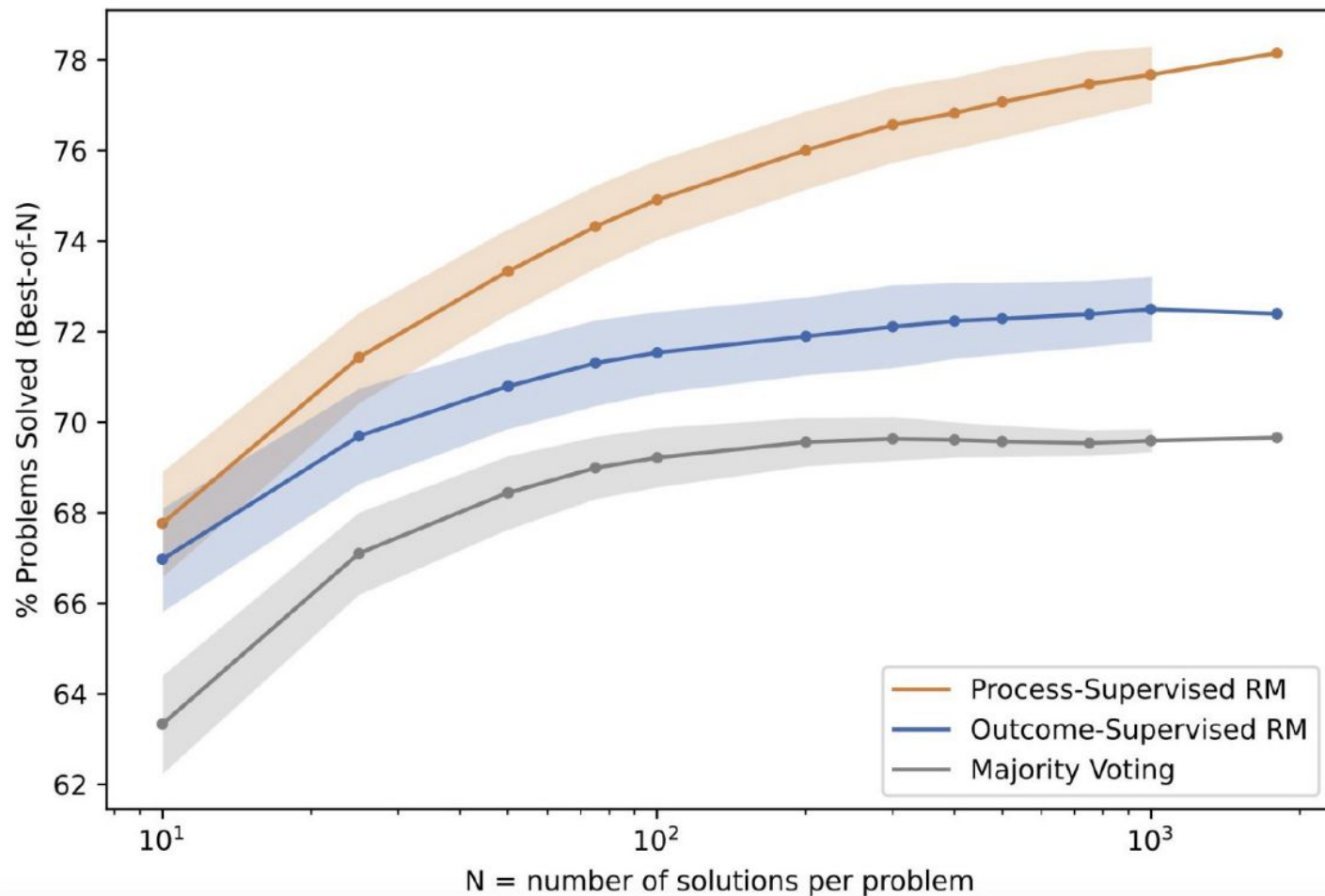


Two types of LLM-based Verifiers / Reward models:

- ❖ Outcome-supervised Reward Model (ORM): Verify at the **(final) solution/ outcome** level
- ❖ Process-supervised Reward Model (PRM): Verify at the **Step-level** for each solution

Best-of-N with Learned Verifier: Results

	ORM	PRM	Majority Voting
% Solved (Best-of-1860)	72.4	78.2	69.6



❖ Process-supervised Reward Model (PRM) scales better with more samples

BUT:

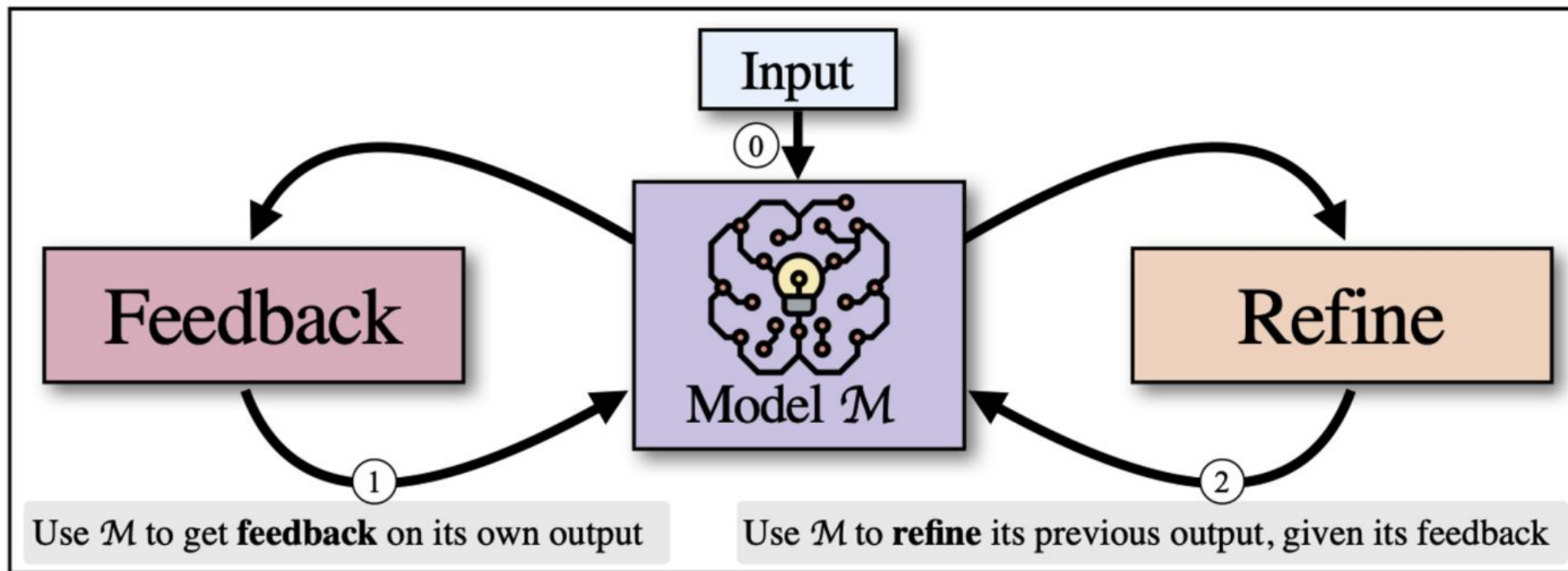
- ❖ Performance highly dependent on verifier quality
- ❖ Same verifier may not generalize across different tasks

Increasing the LLM's Depth of Exploration

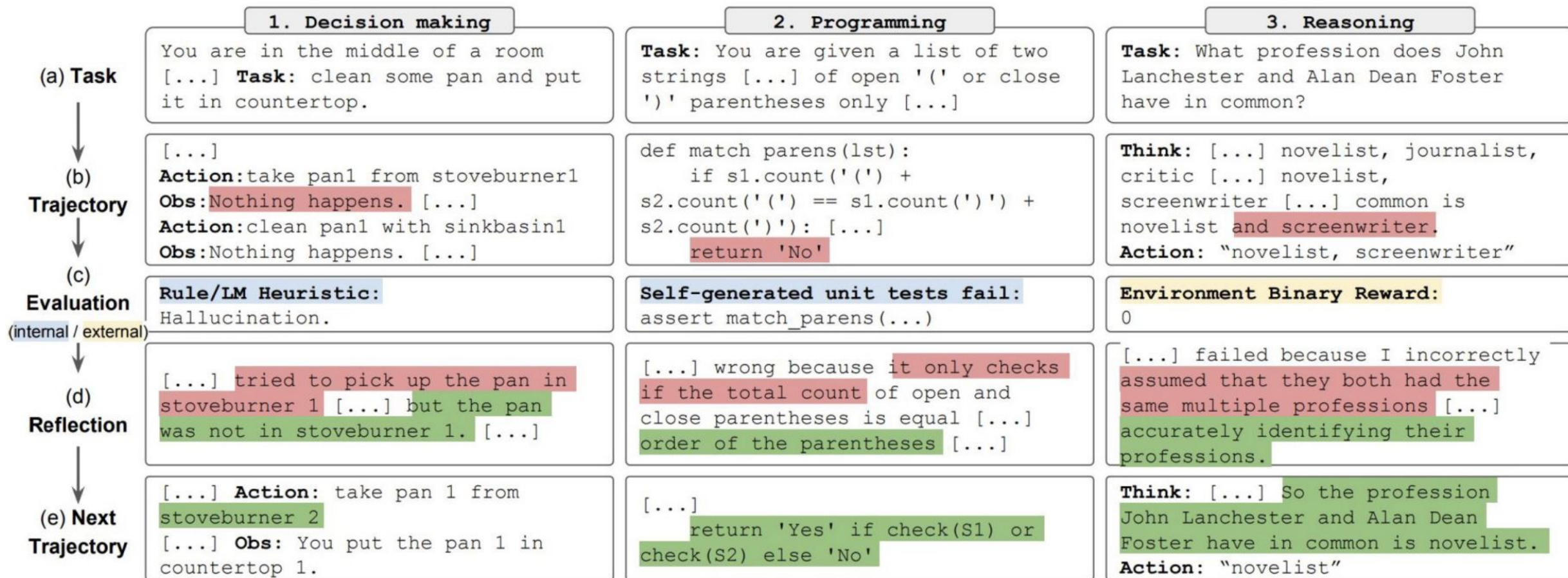
- ❖ Even Humans often make (sometimes trivial) mistakes at first thought
 - ❖ Although sampling multiple solutions (i.e. increase the width of exploration space) can reduce mistakes by considering more than one single prediction, it is still suboptimal because
 - NO feedback loop to correct the mistakes after a complete solution is generated
- In contrast,
- under the Inference-time self-improvement approach, LLM iteratively improves its own response for the given task.

Self-Refine

Basic idea: prompt an LM to critique/revise its own outputs iteratively.



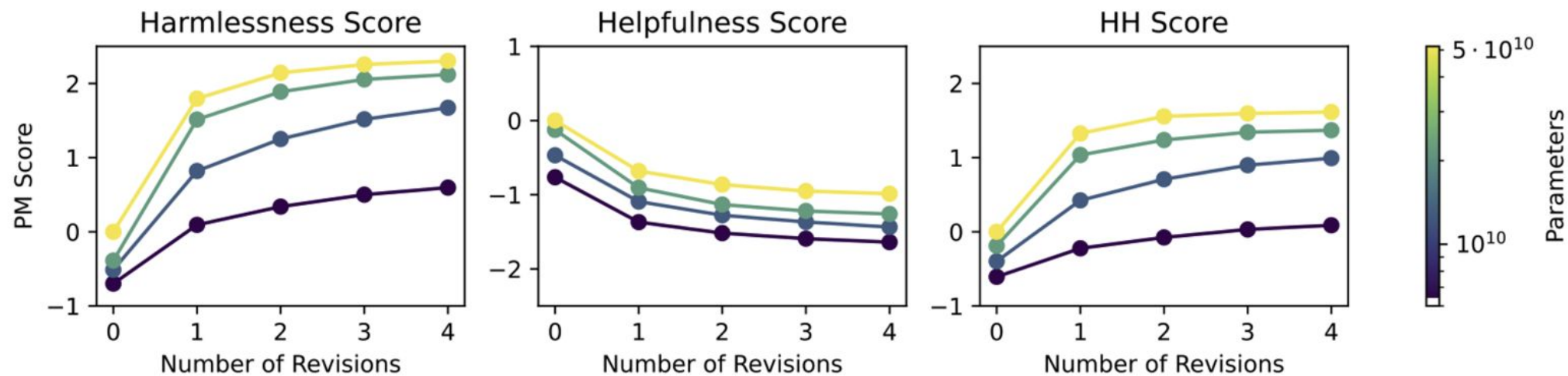
Reflection (Reflexion) and Self-Refine



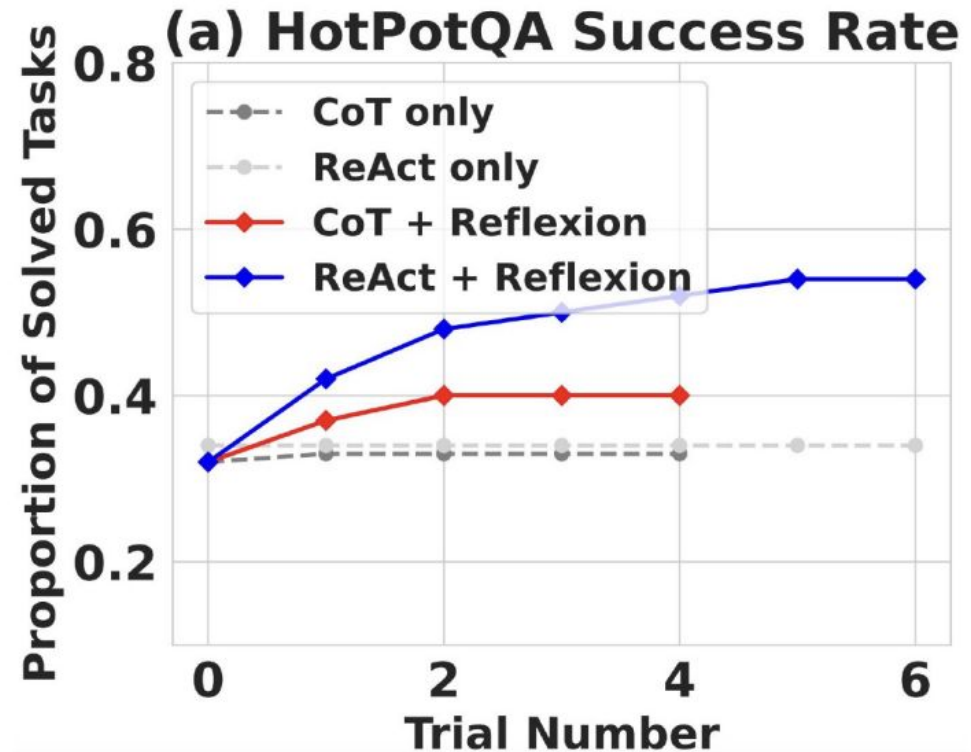
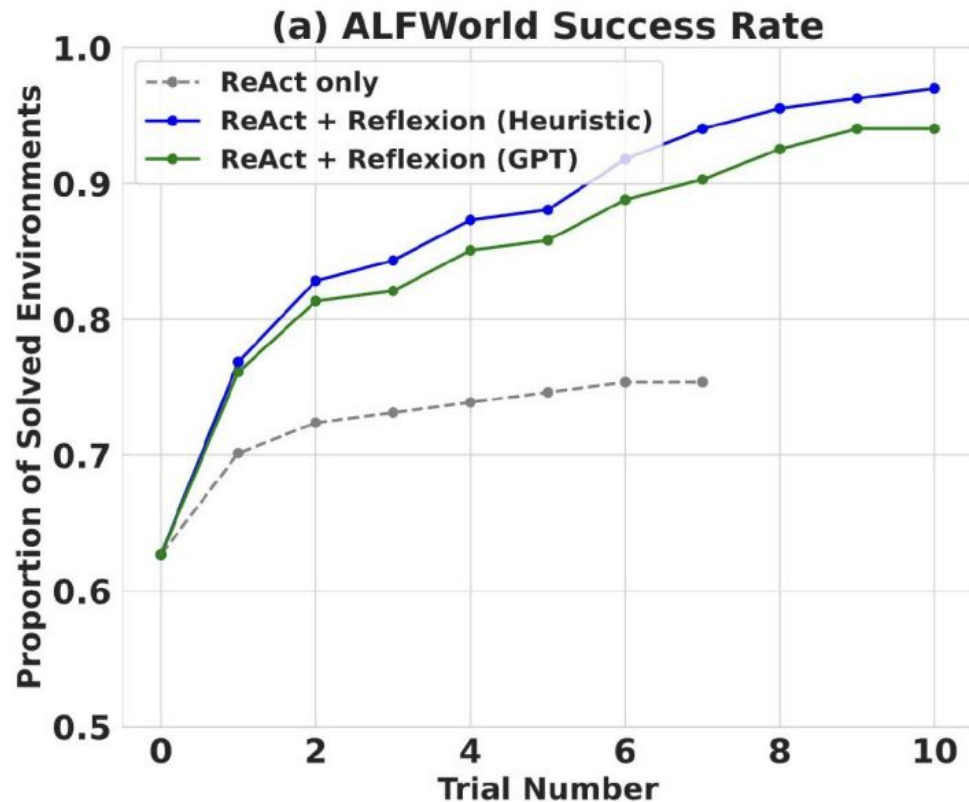
- ❖ LLM generates feedback on its output. Use external evaluation (oracle) when available.
- ❖ LLM self-refines its output given both internal feedback and external evaluation.

Initial Findings

Seems like it works reasonably well on easy tasks like chatbot harmfulness or summarization.



ONLY WHEN Good External Evaluation (Oracle) is available



- ❖ Reflexion improves on tasks with **effective** evaluation heuristics, e.g., ALFWorld.
- ❖ On HotPotQA: **the external evaluation gives the answer correctness** at each reflection step.

Without Oracle feedback hurts Reasoning Performance

		GSM8K	CommonSenseQA	HotpotQA
GPT-3.5	Standard Prompting	75.9	75.8	26.0
	Self-Correct (Oracle)	84.3	89.7	29.0
GPT-4	Standard Prompting	95.5	82.0	49.0
	Self-Correct (Oracle)	97.5	85.5	59.0

		# calls	GSM8K	CommonSenseQA	HotpotQA
GPT-3.5	Standard Prompting	1	75.9	75.8	26.0
	Self-Correct (round 1)	3	75.1	38.1	25.0
	Self-Correct (round 2)	5	74.7	41.8	25.0
GPT-4	Standard Prompting	1	95.5	82.0	49.0
	Self-Correct (round 1)	3	91.5	79.5	49.0
	Self-Correct (round 2)	5	89.0	80.0	43.0

- ❖ Oracle: utilize the ground truth answer for correction
- ❖ Without oracle feedback, LLMs need to judge the response correctness themselves
- ❖ LLMs can wrongly judge the correctness of its predictions, leading to worse performance after self-correction

Oracle available and can be leveraged

Self-debug naturally leverage unit tests as oracle

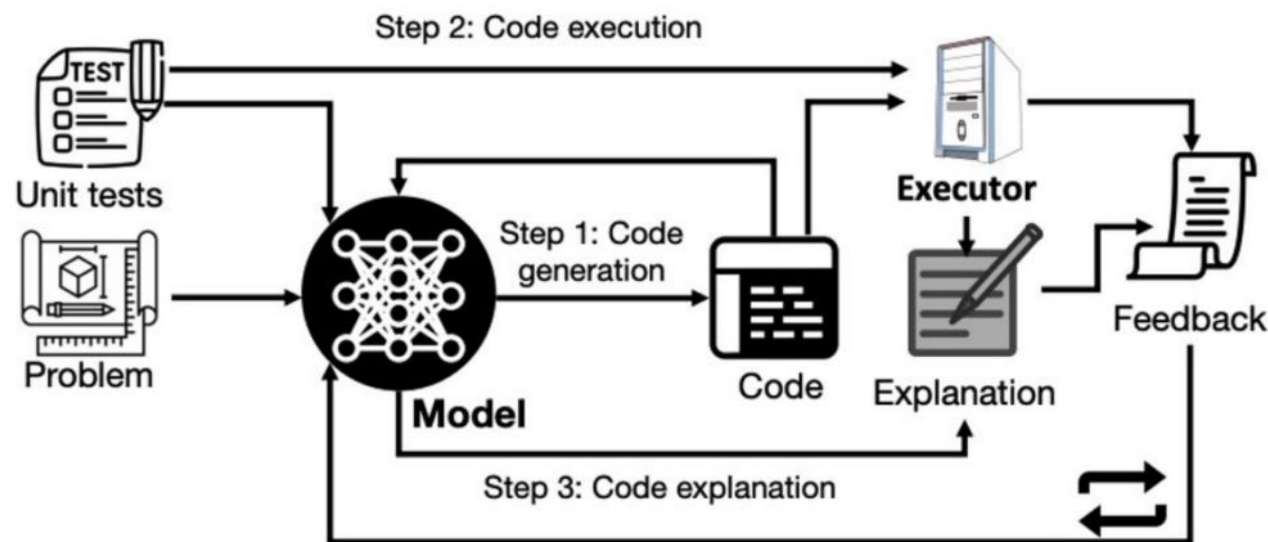


Figure 1: SELF-DEBUGGING for iterative debugging using a large language model. At each debugging step, the model first generates new code, then the code is executed and the model explains the code. The code explanation along with the execution results constitute the feedback message, which is then sent back to the model to perform more debugging steps. When unit tests are not available, the feedback can be purely based on code explanation.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, Denny Zhou. [Teaching Large Language Models to Self-Debug](#). arXiv:2302.00093 [cs.CL], 2023. ICLR 2024.

❖ **Code execution provides natural external feedback:** humans often debug better within an IDE !

Self-Debugging with different Feedback Formats

```
[C++] =
int count_trailing_zeroes_factorial_number ( int
n ) {
    int count = 0;
    for ( int i = 5; n / i >= 1; i *= 5 )
        count += n / i;
    return count;
}
```

```
[Original Python] =
def count_trailing_zeroes_factorial_number(n):
    cnt = 0
    i = 5
    while ((n / i) >= 1):
        cnt = (cnt + (n / i))
        i *= 5
    return cnt
```

[Simple Feedback] =
The above Python translation does not do the same thing as the C++ code. Correct the Python translation.

[UT Feedback] =
The Python translation does not do the same thing as the C++ code. These are the results of failed unit tests that test whether the Python translation's outputs match the C++ program's outputs:

Failed: assert
count_trailing_zeroes_factorial_number(9) == 1
Actual Result: 1.8

Correct the translation.

```
[Revised Python #n] =
def count_trailing_zeroes_factorial_number(n):
    cnt = 0
    i = 5
    while ((n / i) >= 1):
        cnt = (cnt + (n // i))
        i *= 5
    return cnt
```

[C++ Explanation] =
The code is an implementation of counting the number of trailing zeroes in the factorial of a given number.
int count = 0;
The variable count is used to record the number of trailing zeroes.
for (int i = 5; n / i >= 1; i *= 5)
The loop iterates until the quotient of the given number and i (the divisor) is greater than or equal to 1.
count += n / i;
When the loop is executed, the quotient of the given number and i will be added to the variable count.
return count;
Finally, the function returns the number of trailing zeroes.

[Python Explanation] =
The code is an implementation of counting the number of trailing zeroes in the factorial of a given number.
cnt = 0;
The variable cnt is used to record the number of trailing zeroes.
i = 5
The variable i is used as the divisor.
while ((n / i) >= 1):
The loop iterates until the quotient of the given number and i (the divisor) is greater than or equal to 1.
cnt = (cnt + (n / i))
When the loop is executed, the quotient of the given number and i will be added to the variable cnt.
i *= 5
The variable i will be multiplied by 5.
return cnt
Finally, the function returns the number of trailing zeroes.

[Trace] =
1. Initialize 'n' as 9, 'cnt' as 0, 'i' as 5.
2. Enter the while loop since '(9 / 5)' is greater or equal to 1.
3. 'cnt' is assigned 'cnt + (9 / 5)' which evaluates to 1.8.
4. 'i' is multiplied by 5, becomes 25.
5. Check the condition again '(9 / 25)' is not greater or equal to 1. Exit the while loop.
6. Return 'cnt' which is 1.8.

The incorrect translation fails because the division 'n / i' in Python returns a float value, whereas in C++ it returns an integer value. This issue affects the value assigned to 'cnt' and results in an incorrect output.

- ❖ **Simple:** a short universal feedback for all wrong code
- ❖ **Unit test feedback:** include the execution results
- ❖ **Code explanation:** line-by-line explanation of the implementation
- ❖ **Trace:** line-by-line simulation of the execution trace

Self-Debugging Performance

(a) Results on the Spider development set.

Spider	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	81.3	71.1	73.2	64.7
Simple	81.3	72.2	73.4	64.9
+Expl.	84.1	72.2	73.6	64.9

(b) Results on TransCoder.

TransCoder	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	80.4	89.1	77.3	70.0
Simple	89.3	91.6	80.9	72.9
UT	91.6	92.7	88.8	76.4
+ Expl.	92.5	92.7	90.4	76.6
+ Trace.	87.9	92.3	89.5	73.6

(c) Results on MBPP.

MBPP	Codex	GPT-3.5	GPT-4	StarCoder
Baseline	61.4	67.6	72.8	47.2
Simple	68.2	70.8	78.8	50.6
UT	69.4	72.2	80.6	52.2
+ Expl.	69.8	74.2	80.4	52.2
+ Trace.	70.8	72.8	80.2	53.2

- ❖ Self-debugging consistently boosts the performance across different LLMs
- ❖ More informative feedback further improves the debugging performance

Test-time Scaling

Trade-off Analysis of Inference-time (aka Test-time) Scaling:

How to best Utilize the Token Budget for the LLM ?

Type 1 vs Type 2 vs Type 3 approaches to make LLM reason ?

Test-time Scaling

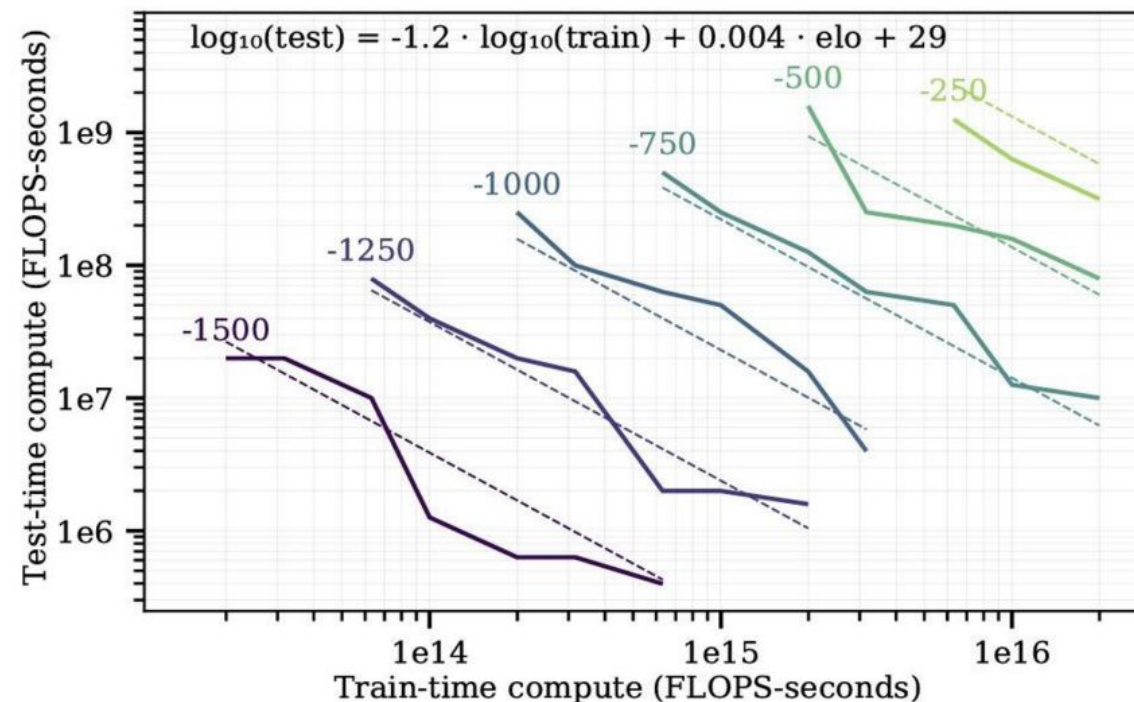
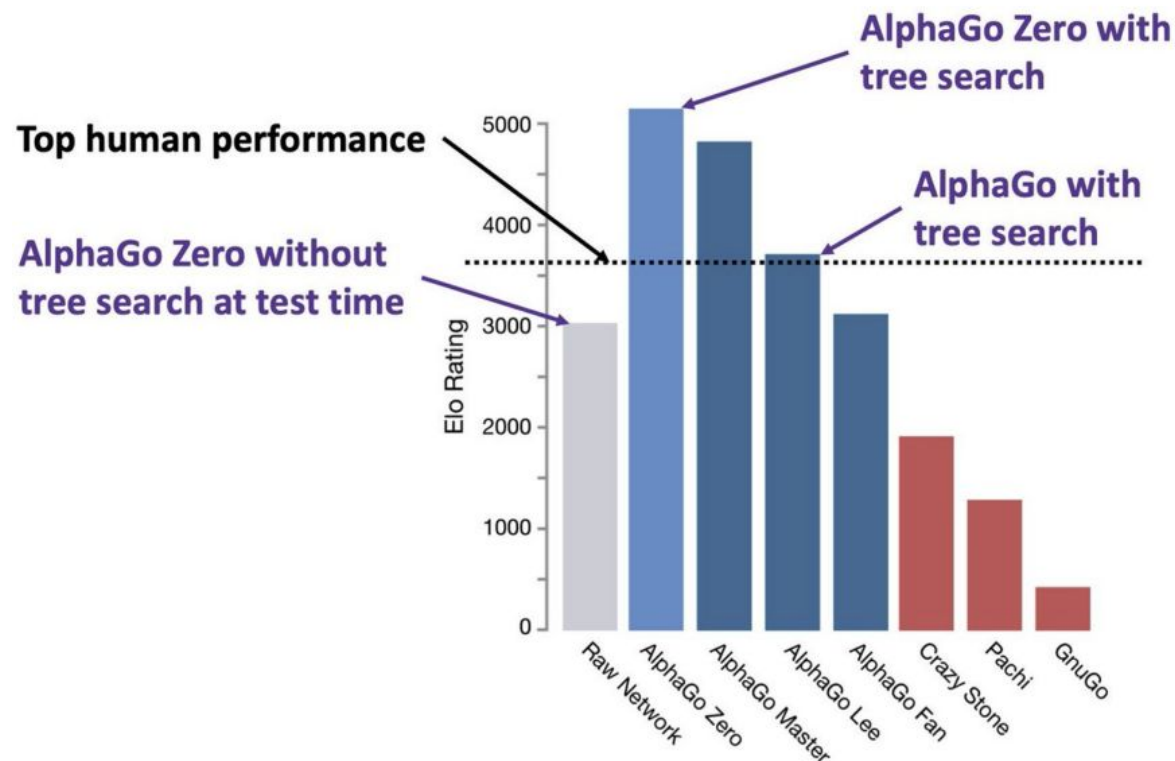


Fig. 9. The trade-off between train-time compute and test-time compute. Each dotted line gives the minimum train-test compute required for a certain Elo on a 9×9 board

- ❖ Giving AI additional test-time compute can greatly improve performance.
- ❖ However, previous demonstrations are mainly limited to specific tasks.

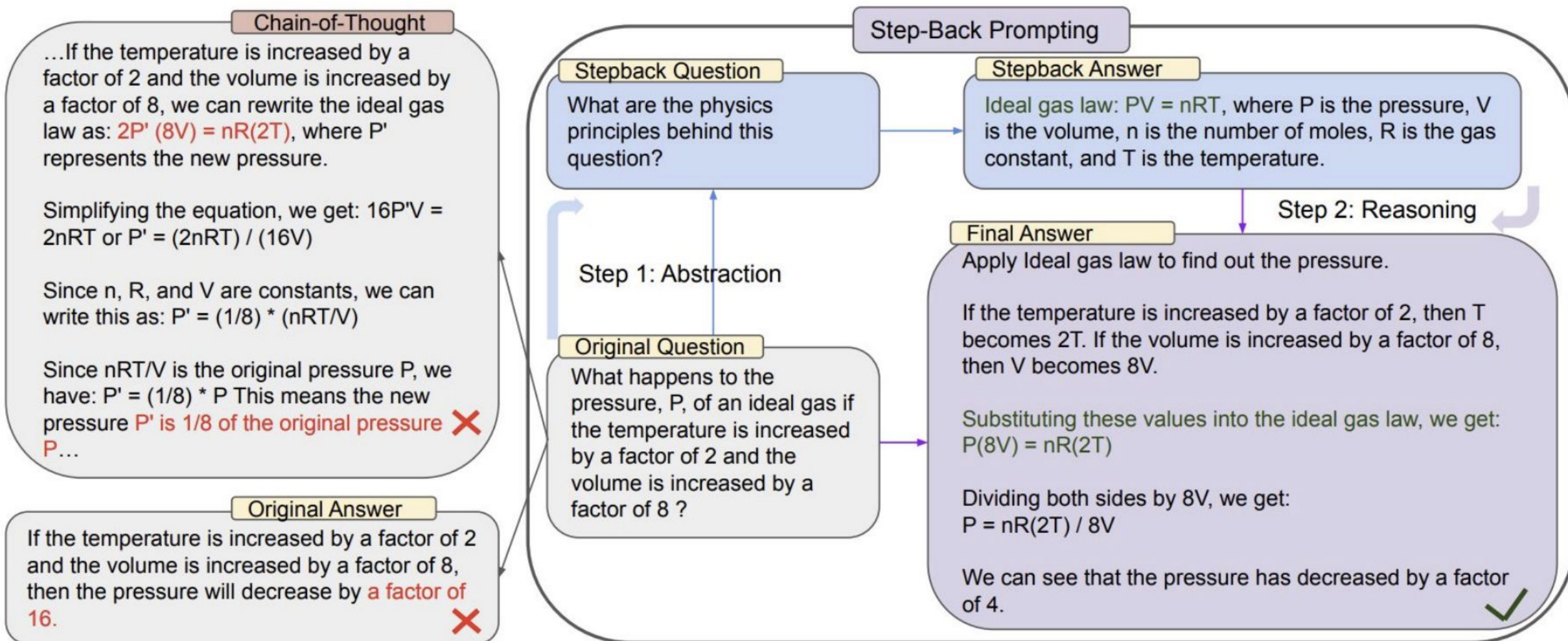
Retrieval + Reasoning

What is the area of the square with the four vertices at $(-2, 2)$, $(2, -2)$, $(-2, -6)$, and $(-6, -2)$? Recall a related problem, and then solve this one.

The related problem is finding the distance between two points on a coordinate plane. To find the distance between two points, we use the distance formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

In this problem, we have four points: $(-2, 2)$, $(2, -2)$, $(-2, -6)$, and $(-6, -2)$. Let's choose $(-2, 2)$ and $(2, -2)$ to find the distance d between them: $d = \sqrt{(2 - (-2))^2 + (-2 - 2)^2} = \sqrt{4^2 + (-4)^2} = \sqrt{16 + 16} = \sqrt{32}$. We know that the distance between two opposite sides of a square is equal. So, the length of each side of the square is $\sqrt{32}$. To find the area of the square, we square the length of one side: $\text{Area} = (\sqrt{32})^2 = 32$.

Retrieval + Reasoning



Deep Research

Gemini Deep Research



Search



Deep research



ChatGPT



Search



Deep research

Ways to Build or Improve Reasoning

1) Inference-time compute scaling

] **test-time compute**

2) Pure RL (e.g., DeepSeek R1 Zero)

3) RL + SFT (e.g., DeepSeek R1)

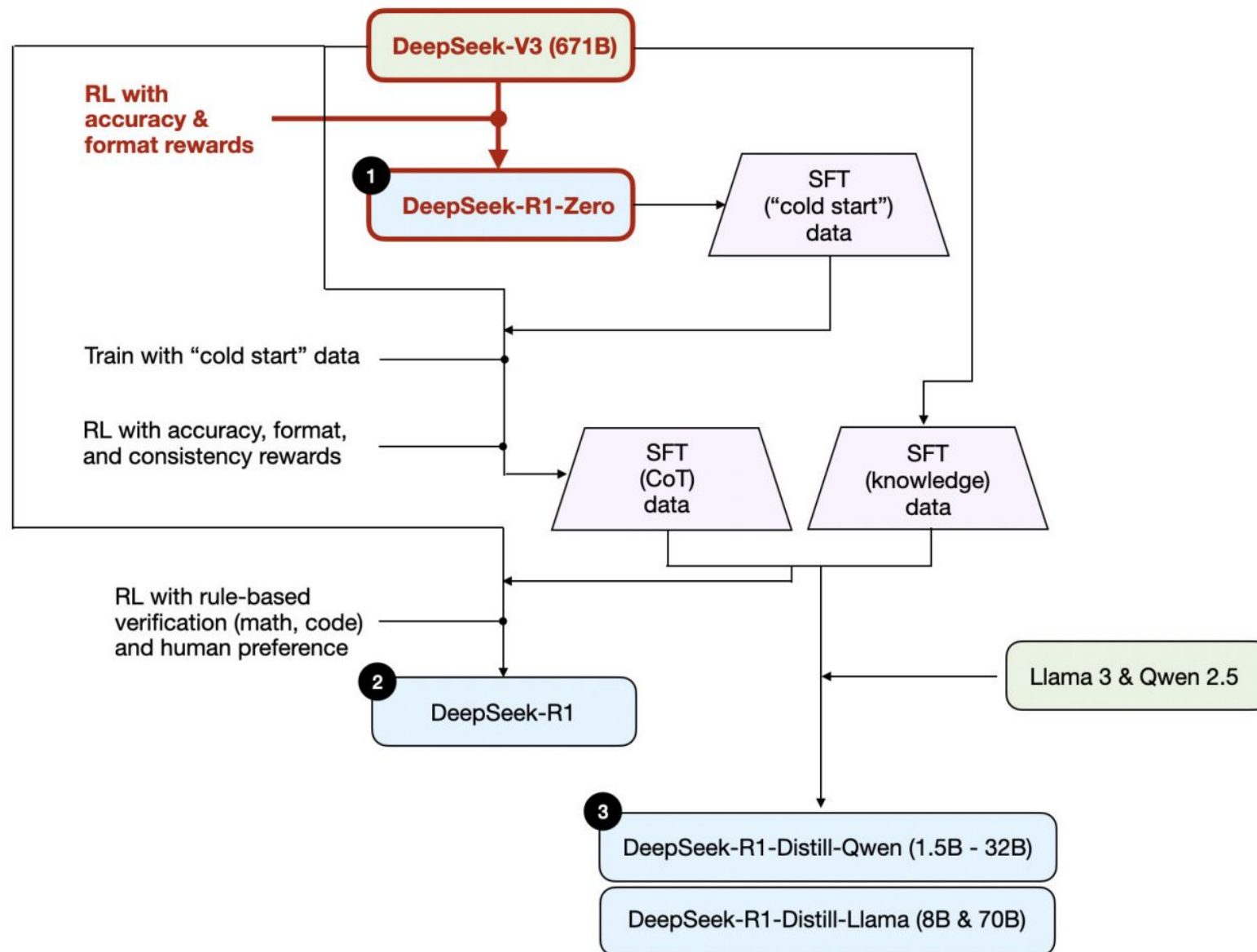
4) pure SFT (e.g. s1, distilled DeepSeek R1)

] **train-time compute**

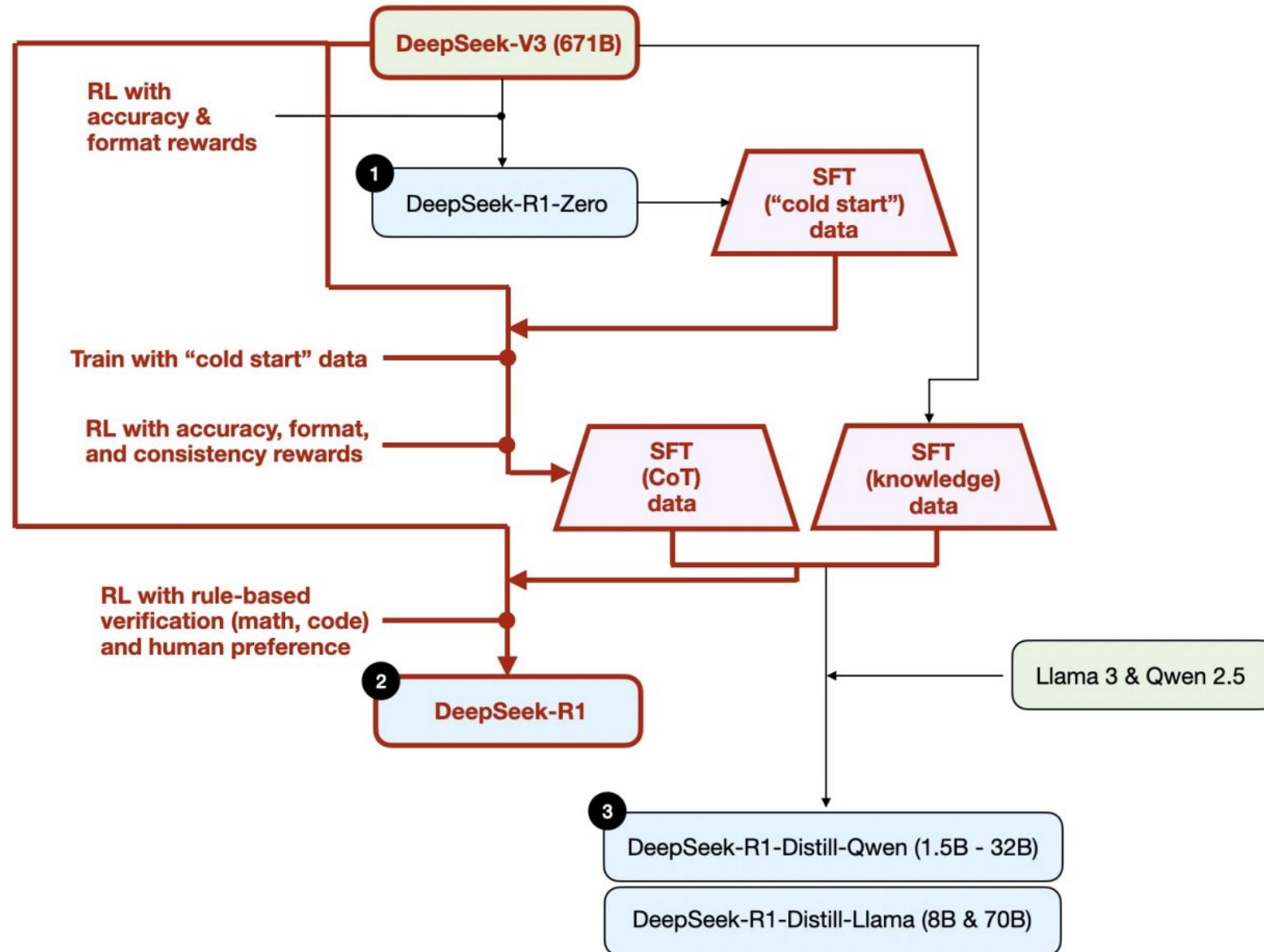


Sebastian Raschka

“Pure” Reinforcement Learning



Supervised Fine-Tuning (SFT) + RL



Performance Comparisons

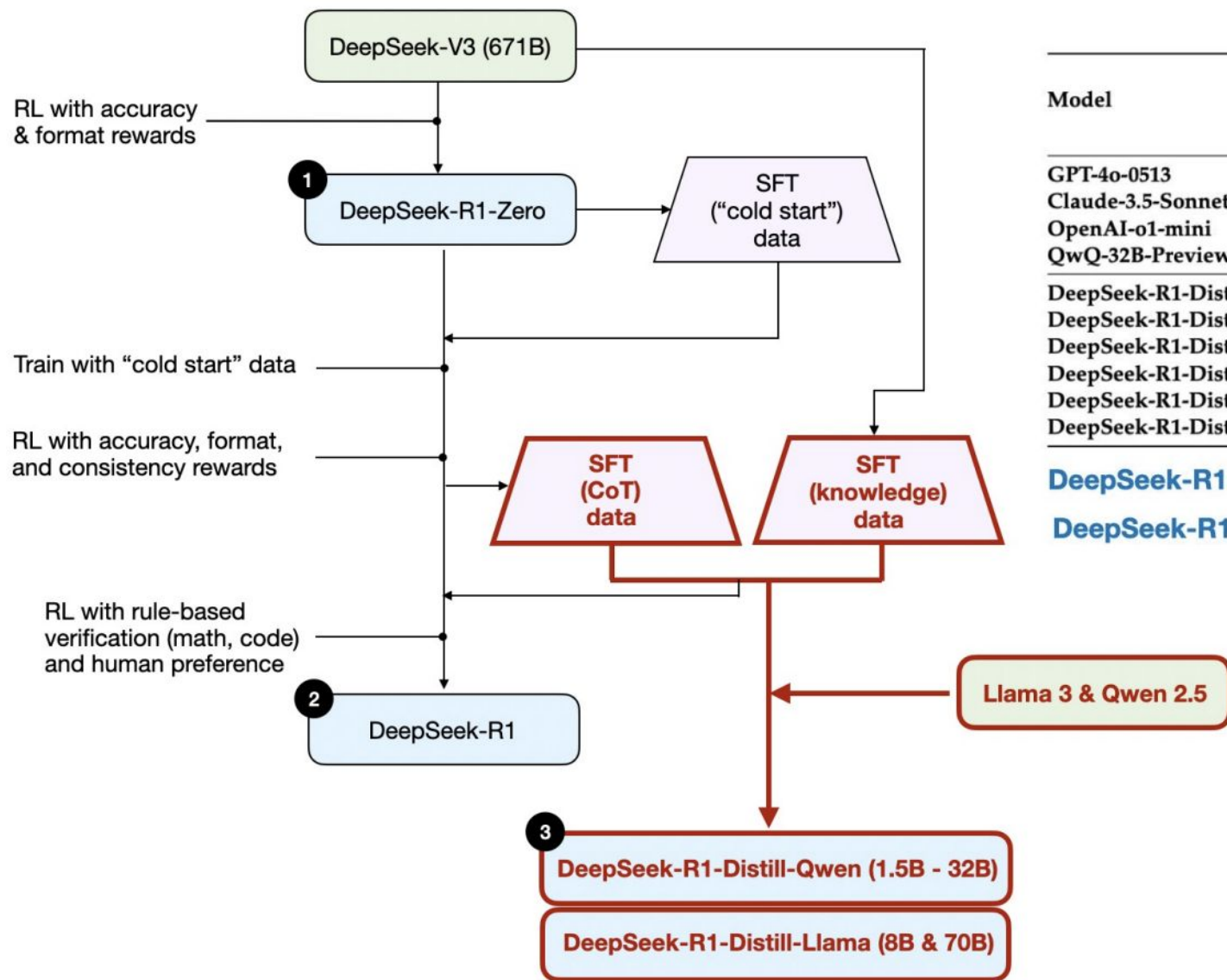
Model	Math benchmarks			Bio, physics & chemistry	Code benchmarks	
	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444
DeepSeek-R1	79.8		97.3	71.5	65.9	2029

Higher is better

RL only →

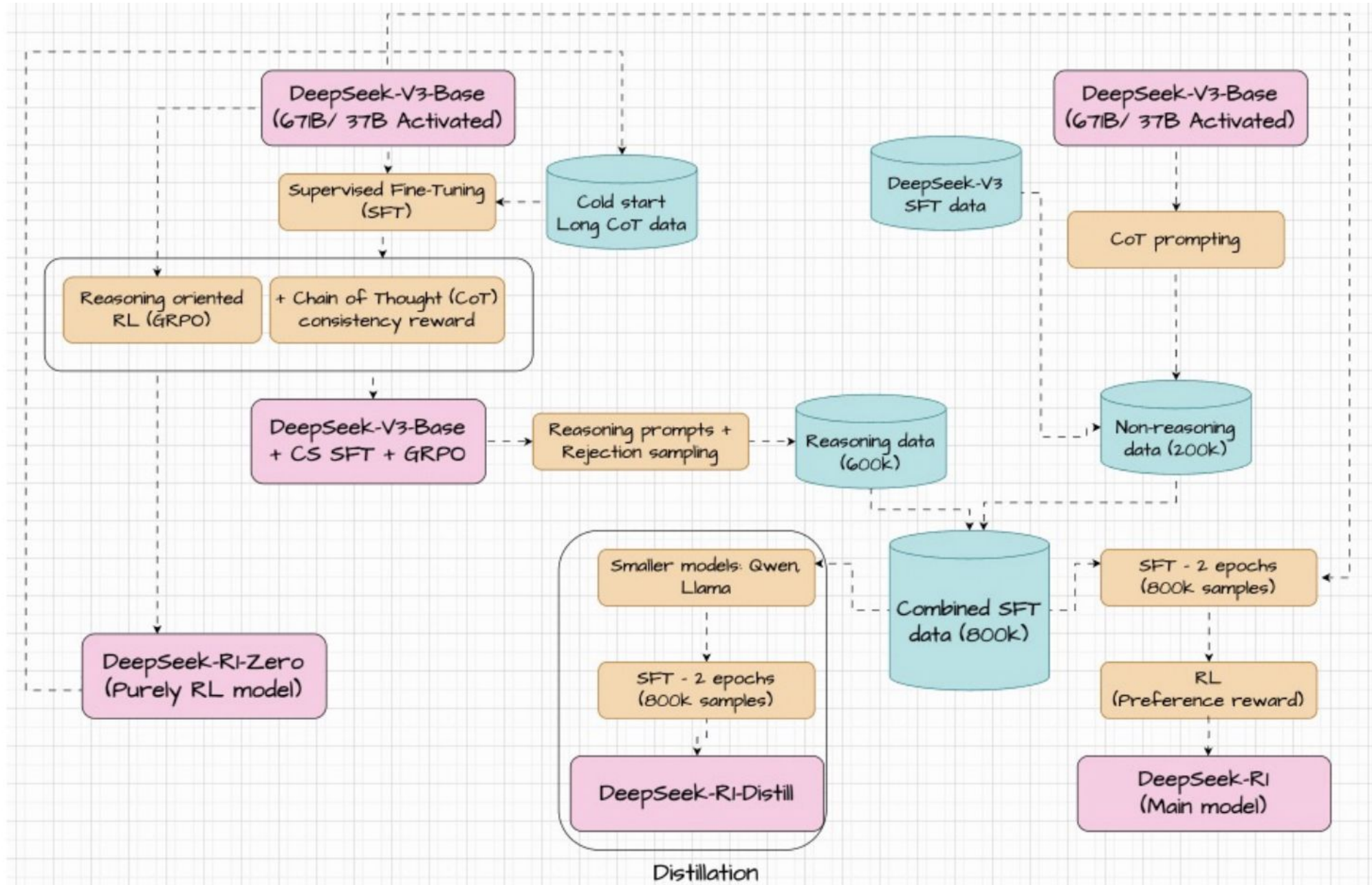
SFT + RL →

SFT + Distillation



Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
QwQ-32B-Preview	50.0	60.0	90.6	54.5	41.9	1316
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	72.6	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	86.7	94.5	65.2	57.5	1633
DeepSeek-R1-Zero	71.0		95.9	73.3	50.0	1444
DeepSeek-R1	79.8		97.3	71.5	65.9	2029

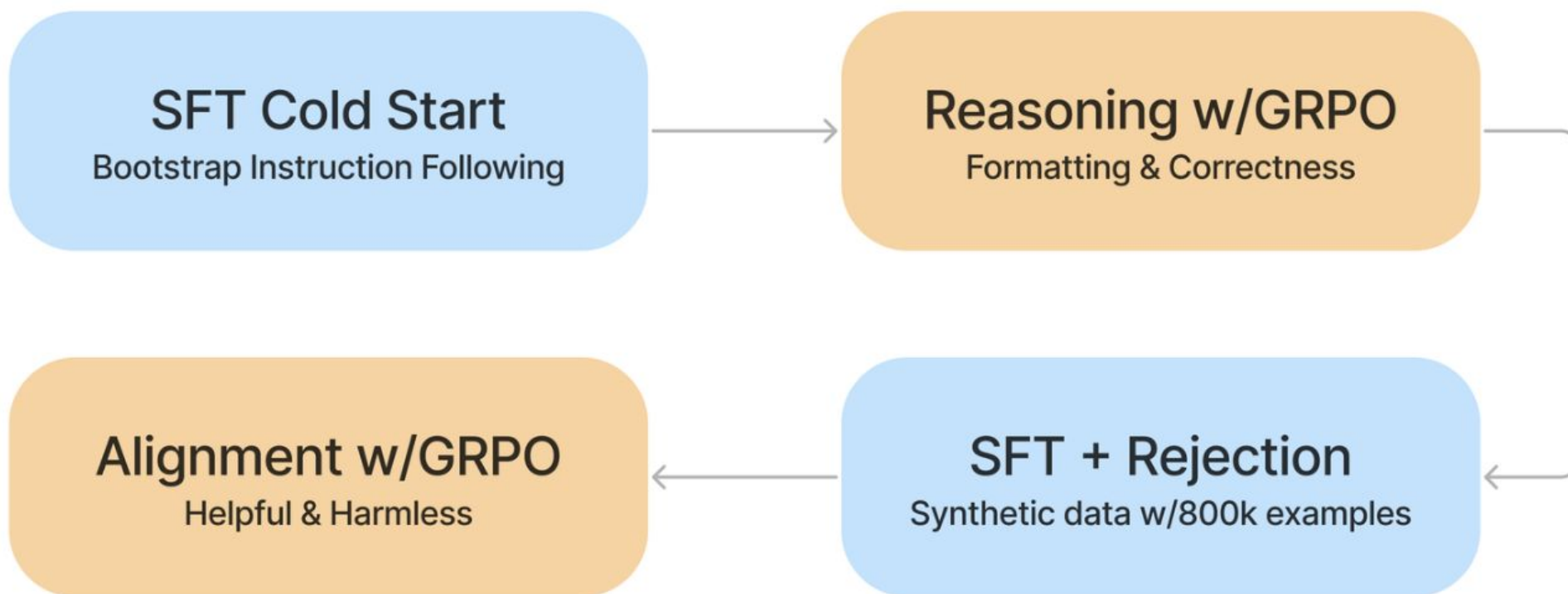
More Details



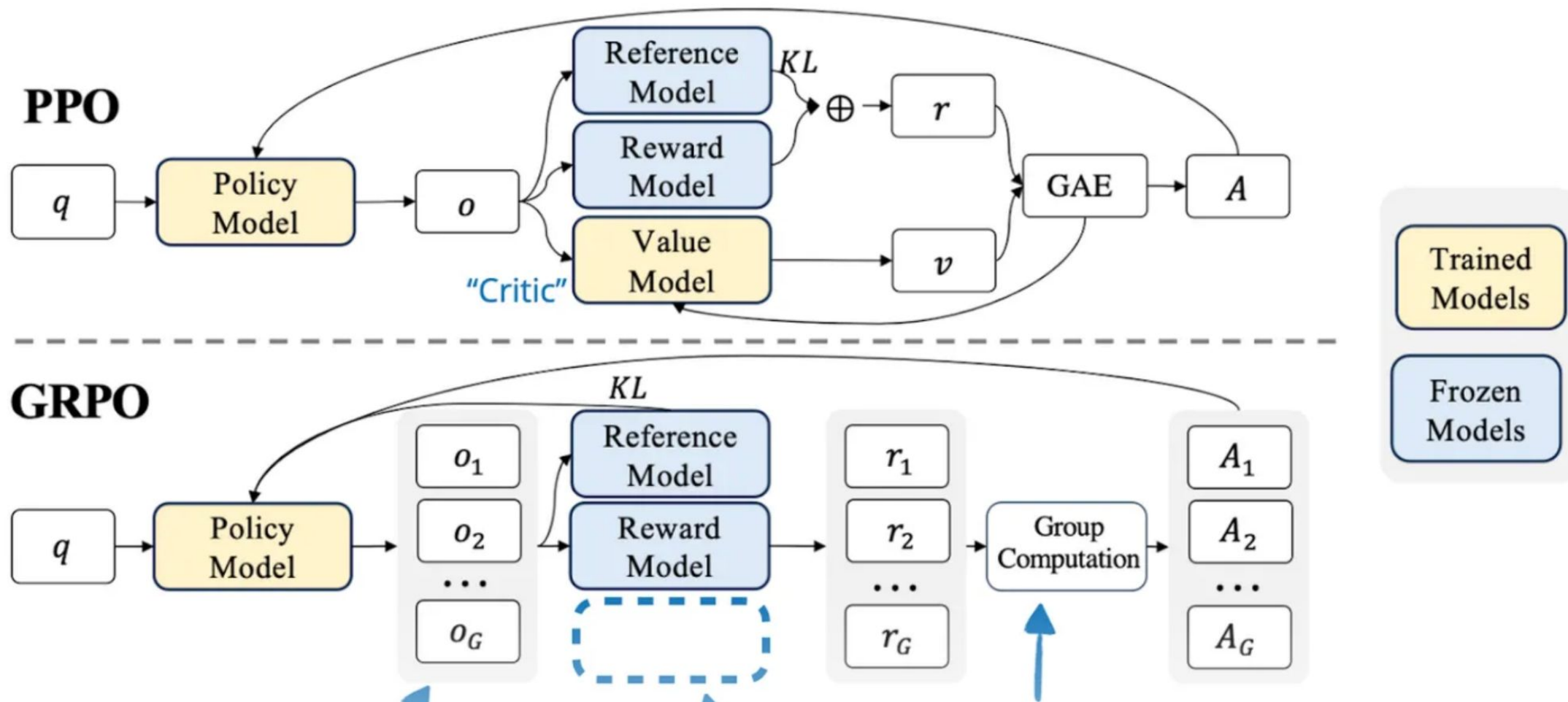
Why GRPO?



DeepSeek-R1 Training Pipeline



Why GRPO?



Thanks

Q&A

Prof. Yu Cheng
chengyu@cse.cuhk.edu.hk

